Getting Started with Propeller 1 Plugins

Basic Concepts

One of the design goals of Catalina was to make the environment in which C programs execute as platform-independent as possible. Catalina does this partly by implementing an effective *Hardware Abstraction Layer* (HAL) so that most C programs do not need to know the details of the drivers that perform various platform-dependent I/O functions on their behalf.

On the Propeller, with its multi-core capabilities, drivers are usually implemented in a separate cog to the main program. But as well as device drivers, the multi-core capabilities of the Propeller encourage the use of cogs for performing other functions as well – such as floating point functions, or various library functions that would be inefficient to code in C.

Catalina uses the term ***Plugin*** to refer to such components – i.e. a program designed to run on a separate cog. This term was chosen to highlight the fact that plugins are usually developed independently of the programs that make use of them, that they may be used by many programs, and also that they may be dynamically loaded as needed and unloaded again when no longer required (although many device driver plugins are loaded once at initialization time and never unloaded).

Catalina defines a common technique for keeping track of all the loaded plugins, and also a common method for communicating with those plugins. This is done by using the ***Registry*** – which is a just section of Hub RAM reserved for plugins. However, the registry is not usually accessed directly by application programs – a set of low level registry functions to do so are provided, but these are usually "wrapped" in a set of user-friendly C functions that are plugin-specific. For example, to access a Real-Time Clock plugin, a function such as *get_time()* would typically be provided, which implements all the low-level work of interacting with the clock plugin via the registry.

Finally, Catalina introduces the concept of a ***Target***. A target defines the execution environment for a Catalina C program – including the hardware configuration (pin definitions, physical addresses, clock speeds) the software configuration (drivers and plugins) and the kernel itself (LMM or XMM kernel[1], or one of the special function kernels such as the multi-threading kernel). Catalina groups all these components together into a ***Target Directory*** (sometimes called a ***Target Package***) for convenience.

This document specifically discusses Targets and Plugins for the Propeller 1. The Propeller 2 Targets and Plugins are similar in concept and design, but differ in internal detail.

Each target directory usually supports one or more Propeller platforms. For example, the standard target directory provided with Catalina for the Propeller 1 supports the *Hydra*, *Hybrid*, *TriBladeProp*, *DracBlade*, *RamBlade*, *Demo* and *C3* boards and the *Flip* module. Each target directory will also contain a specific set of plugins supported by those platforms (e.g. the standard Target Directory includes various *HMI*, *SD card*, *Real-Time Clock*, and *Floating Point* plugins).

Catalina allows multiple target directories, but only one can be specified when compiling the final program executable. Having multiple target directories can come

[1] For detailed information on the various Catalina Kernels and memory models (e.g. LMM, EMM, XMM etc) see the **Catalina Reference Manual**.

in handy when developing new plugins, or when supporting many different propeller platforms with significantly different capabilities. This will become more evident later in this tutorial, since we will use a separate target directory to illustrate how to build a plugin – without affecting the standard Catalina Target Directory.

Within each target directory, there may be support for different individual targets. Each target specifies the kernel to be loaded, and each target knows how to load and initialize all the plugins supported by that target. For example, in the standard target directory there are LMM targets, EMM targets, XMM targets, and debug targets[2].

This last point is quite important – this is because even though a plugin is essentially a "stand-alone" program, they can be complex to load and initialize correctly. Often plugins must share not only the Registry, but sometimes other areas of Hub RAM, or other Propeller resources such as I/O pins or locks. It is the individual *targets* that know how to load and initialize the various plugins correctly – especially since the load process can differ depending on the type of kernel used. For example, LMM programs are loaded quite differently to EMM programs or XMM programs.

In summary, developing a plugin to be used in conjunction with a Catalina C program consists of three parts:

1. The plugin itself. Plugins are usually implemented in PASM (although they can also be written in Spin), and are often adapted from existing code.
2. A set of C "wrapper" functions that allows the services provided by the plugin to be more easily invoked from C.
3. One or more targets that support the plugin (i.e. that know how to load it). This is most easily accomplished using the **Extras.spin** file in each target directory (see **Setting Up and Starting Plugins** later in this document).

This document provides a brief overview of all these aspects of developing Catalina plugins. However, It is not a "step-by-step" or "hands-on" tutorial. Instead, it simply discusses various aspects of the process - but a fully documented example of a working "generic" plugin – plus some C wrapper functions that show how to invoke it, plus a Catalina target that loads it, plus some demo programs that use it – are all provided, and a referred to in various places in this document.

The remainder of this document also assumes you are at least slightly familiar with the C language, SPIN, PASM and also with the Propeller architecture.

This document assumes you are creating a plugin for the Propeller 1. The steps are similar to the Propeller 2, but there is as yet only one supported Propeller 2 Target (*target/p2*).

Configuring the minimal target

All the files you need will already be installed in two subdirectories in the main Catalina directory – typically *C:\Program Files (x86)\Catalina* (Windows) or */opt/catalina* (Linux).

[2] For the Propeller 2 there is also an NMM target.

There are two important subdirectories:

minimal\p1 This is a complete – but *minimalist* – Catalina Target Directory (also called a Catalina Target Package). It contains a single LMM *target*, and a single *plugin* that this target knows how to load. The LMM target is called **default**, which means it does not normally need to be specified on the Catalina command line. The plugin is defined in the file *Catalina_Plugin.spin*, and is enabled by defining the symbol **PLUGIN** on the Catalina command line.

demos\minimal This directory contains a set of example wrapper functions (defined in *generic_plugin.h* and implemented in *generic_plugin.c*) that allow easy access to the services provided by the example plugin. It also contains a couple of demonstration programs that use them.

In each of these directories (and their subdirectories) you will find *README.TXT* files, which contain detailed information in addition to what is contained in this tutorial.

Before you can compile the demonstration programs, you may first need to go to the *minimal\p1* subdirectory and edit the *Custom_DEF.inc* file (make sure you edit the version in the *minimal\p1* subdirectory, and not the one in the normal Catalina *target\p1* directory) and specify your own platform details (and also comment out the **#error** line).

Then go to the *demos\minimal* subdirectory, and use the following command to build all the example programs:

```
build_all
```

Editing the *Custom_DEF.inc* file to suit your propeller platform is quite trivial – the contents of the file will appear similar to the following (use any text editor to view it):

```
'
'
' Throw an error if this file has not yet been edited
' (remove the following line after editing):
#error The Custom_DEF.inc file needs editing!
'
'=============================================================================
'
' CUSTOM platform General definitions:
'
'=============================================================================

KBD_PIN    = -1                ' BASE PIN  (Custom)
MOUSE_PIN  = -1                ' BASE PIN  (Custom)
TV_PIN     = -1                ' BASE PIN  (Custom)
VGA_PIN    = -1                ' BASE PIN  (Custom)
SD_DO_PIN  = -1                ' Custom has no SD Card
SD_CLK_PIN = -1                ' Custom has no SD Card
SD_DI_PIN  = -1                ' Custom has no SD Card
SD_CS_PIN  = -1                ' Custom has no SD Card
I2C_PIN    = 28                ' I2C Boot EEPROM SCL Pin
I2C_DEV    = $A0               ' I2C Boot EEPROM Device Address
SI_PIN     = 31                ' PIN (Custom)
SO_PIN     = 30                ' PIN (Custom)
'
' Custom platform Clock definitions:
'
CLOCKMODE = xtal1 + pll16x     ' (Custom)
XTALFREQ  = 5_000_000          ' (Custom)
CLOCKFREQ = 80_000_000         ' (Custom) Nominal clock frequency
                               ' (required by some drivers)
```

For the purposes of this tutorial, all you need to do is set the clock frequency information correctly, and remove (or comment out) the **#error** line. You don't need to modify any pin definitions, because neither the demo programs nor the generic plugin use them.

The Generic Plugin

Examine the file *Catalina_Plugin.spin* in the *minimal\p1* subdirectory. It embodies many of the aspects common to all PASM plugins, and the rules that all plugins must follow:

- It contains a CON section. However, note that any constants defined here can only be used within the plugin itself, or within the target that loads it. If any of these constants are required in C, they will need to be redefined in a C header file.
- It contains an OBJ section which includes **Catalina_Common**, which in turn includes **Custom_DEF.inc**. This is where all common platform dependent information (such as I/O pin definitions and clock speeds) will be stored – such things should not be duplicated in the plugin itself.
- It contains no SPIN methods, other than **setup** and **start** methods which will be invoked by the target at load time (but note that if the plugin is to be re-loaded dynamically, these methods cannot be called, but its functionality will have to be duplicated in C).
- It contains a VAR section, which is used only during initialization. After initialization any variables defined in the VAR block would no longer exist in Hub RAM at run-time. Instead, a plugin that needs Hub RAM should be told what Hub RAM to use dynamically – either during startup (i.e. in the **start** method) or later by passing initialization parameters via the registry. Doing this allocation statically at initialization time is ok for plugins that are never intended to be unloaded or reloaded, but in this case the registry technique is used since we want to be able to reload and restart the plugin dynamically. However, this does not mean that we have to wait and initialize the plugin from C – the **start** method of this plugin actually uses the registry to initialize the plugin at load time.
- Now look at the DAT section. This contains the PASM that implements the plugin. The first thing the plugin does (see the code at label **entry**) is register itself. This is sometimes done in the start method, but for plugins that may be loaded dynamically, it is better if they register themselves on startup. All plugins are passed the address of the registry on startup – this will appear in the **par** register. From this, they use their cog id to calculate their registry block address that is used for all subsequent communications. See the next section for more details on registration.
- This plugin accepts various service requests, so the next part of the code is about waiting for a service request to appear in the registry (see the code at label **get_service**). In this case, the plugin accepts four different service requests. By convention, the service request code is put in the upper 8 bits of the registry request block (note that plugins do not *need* to accept any service requests). See the next section for more details on using the registry.

- Not all plugins simply wait idly for service requests. Many spend most of their time doing other things (in the example, see the stub code at label **do_stuff**). However, all plugins that accept service requests should check periodically, since the kernel will halt when a service request is made until the request is acknowledged by the plugin.
- At the end of processing each service request, (or earlier, if the request is to be processed asynchronously) the plugin should acknowledge receipt of the request by writing zero to the same location (see the code at label **done_service**). See the next section for more details on using the registry.
- A service request can be invoked using a *short* request or a *long* request. However, this must be fixed at design time since the interpretation of the registry is slightly different for each kind of request, and a service cannot accept both. In this example, service 1 and service 2 are short requests (they can accept up to 24 bits of data), service 3 is a long request (it accepts 32 bits of pin data) and service 4 is a long request (it accepts a 32 bit pointer). Service 4 also illustrates how more than one 32 bit value must be passed – i.e. by passing an address of a temporary memory block that contains the actual parameters. See the next section for more details on the registry.
- A service that must return a result can also return it via the registry. See the next section for more details on using the registry.

Note that the plugin is intended mainly as an example – but it does implement some trivial functions – i.e. it toggles I/O pins. This means that it is possible to see the plugin in action when using the demo programs provided – e.g. on the Hydra or Hybrid, the operation of the plugin can be verified because it will toggle the Debug LED on or off.

The Registry

At its simplest, the Propeller 1 Catalina Registry is simply 8 consecutive longs – one per cog[3] - located somewhere in Hub RAM (usually in the upper area of Hub RAM). The location is fixed only at compile time, and it is quite feasible for two different Catalina programs to use two different locations for the Registry (provided they do not run at the same time).

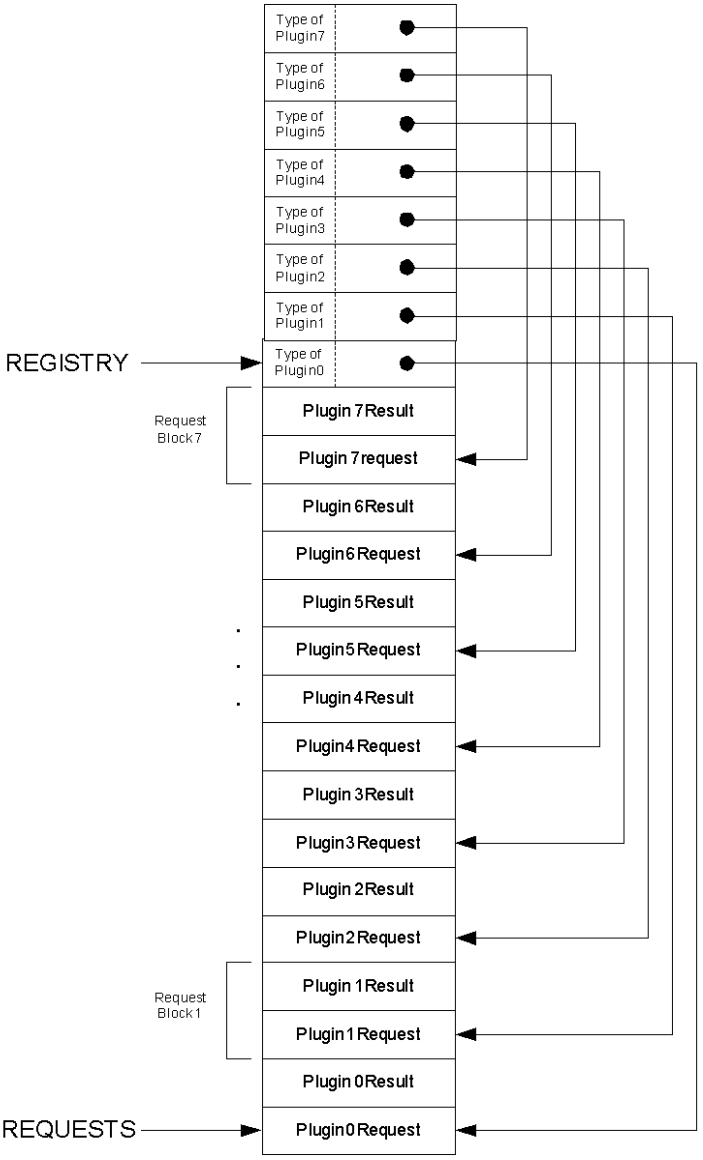
Each plugin is told the location of the registry on startup, and from that (and its own cog id) deduces which of the 8 longs belongs to it specifically. This is referred to as the plugins **Registry Entry**. The Registry Entry is used for two purposes:

1. The upper 8 bits of the Registry Entry are used to indicate the type of plugin loaded into each cog. The value $FF indicates no plugin is loaded (although this is not a guarantee that the cog is actually unused – it simply means no plugin is registered in that cog). Setting this value is referred to as “registering” the plugin.
2. The lower 24 bits of the Registry Entry point to another location in Hub RAM – this is the plugin **Request Block**, and must consist of at least two consecutive longs. In theory, the Request Block can be longer than two longs, can exist anywhere in Hub RAM, and can be set up anytime up to the time the plugin is loaded – but it cannot usually be changed *after* the plugin is loaded, since

[3] The Propeller 2 Registry is 16 longs, in anticipation of a 16 cog Propeller 2,

most plugins read this location into a local cog register during initialization and thereafter do not refer to the Registry Entry. However, it is so common for plugins to use a simple Request Block consisting of two longs, that Catalina initializes each Registry Entry to point to a Request Block of two longs that also lives permanently in upper Hub RAM.

The basic Registry structure is defined in *Catalina_Common.spin*. If you look in this file, you will see definitions for two important locations – **REGISTRY** and **REQUESTS**, and a method called **InitializeRegistry**. After the registry has been initialized, its structure will be as shown in the diagram below.



However, there is nothing to stop a target (or a C program) creating a new request block if it decides it needs one larger than 2 longs – all it has to do is update the Registry Entry prior to loading the plugin.

When using the registry to communicate with a plugin, a program writes a *request* to the *first* long in the plugin request block. A plugin that is offering interactive services must monitor this long, and process the service request whenever it sees a non-zero

value in this long. To indicate the request is complete (or simply to acknowledge receipt of the request), the plugin should write zero to the same long (i.e. the *first* long of the request block). Before doing so, if it wishes to return a result (or a status), the plugin should write a "result" value to the *second* long of the request block.

As has been mentioned, there are several types of service request, dictated by the convention (and it is *only* a convention – plugins are free to use other methods) that plugins offering multiple services use the upper 8 bits of the request long as a service code, and the lower 24 bits of the request long in one of two ways:

- To hold up to 24 bits of data. This is known as a *short* request
- To hold the pointer to *another* data block somewhere in Hub RAM that contains the actual data. This is known as a *long* request. It is the responsibility of the caller to allocate the necessary space for this additional data block, and to guarantee that it remains valid for the lifetime of each service request.

As can be seen from examining the code, short requests are simpler and more efficient, and are generally preferred where possible.

Catalina provides C functions for locating the registry, registering and unregistering plugins, and for sending *short* or *long* service requests to a plugin. They are defined in the file *catalina_plugin.h* in the *Catalina\include* directory. The implementation of these functions can be found in the library source code in *Catalina\source\lib\catalina.*

Here is a summary of the registry-related functions and macros defined in *catalina_plugin.h*:

`unsigned _registry();`

This function returns the address of the registry. This is required to be passed to a cog when starting a dynamic kernel to execute C code on that cog.

`void _register_plugin(int cog_id, int plugin_type);`

This function can be used to register that a plugin of a specified type is running on a particular cog. Plugins must be registered before requests can be sent to them.

`void _unregister_plugin(int cog_id);`

This function can be used to unregister a plugin.

`int _locate_plugin(int plugin_type);`

This function can be used to find a cog on which a plugin type is executing. Note that if there is more than one plugin of a specified type executing, only the first will be found.

`REGISTRY_ENTRY(i)`

This macro returns the registry entry for cog i. The value of i should be from 0 and 7 – any other value will return an undefined result. The result is an unsigned value.

`REGISTERED_TYPE(i)`

This macro returns the registered type for cog i. The value of i should be from 0 and 7 – any other value will return an undefined result. The result is an unsigned value.

REQUEST_BLOCK(i)

This macro returns a pointer to the request block reserved for cog i. The value of i should be from 0 and 7 – any other value will return an undefined result. The request block structure pointed to is defined as:

```
typedef struct {
   unsigned int request;
   unsigned int response;
} request_t;
```

For an example of a program that uses these functions, see the program *test_plugin_names.c* in the main *Catalina\demos* folder (but note that this program cannot be compiled and run using the **minimal** target, as this target has no input/output plugins!).

Loading a plugin dynamically is usually as simple as using the *_coginit()* function and then registering the plugin (if it does not do so itself), and stopping it is usually simply a matter of calling *_cogstop()* on the cog in which it is running, and then unregistering it.

For the “generic” plugin provided (which accepts fours different service requests) a set of “wrapper” functions are defined in the *demos/minimal* directory in the file *generic_plugin.h* (along with other plugin-specific constants that may be required), and implemented in the file *generic_plugin.c.*

It is recommended that these “wrapper” functions be compared with the implementation of each of the services in the file *Catalina_Plugin.spin*. In particular, note that the type of service (i.e. *short* or *long*) must match between the wrapper function and the service implementation.

The Target

The Propeller 1 Targets in the standard Catalina Target Directory (the subdirectory called *target/p1*) are very complex – but this is largely because they each support so many different plugins, kernel types and platforms.

In order to show that the basics of a fully functional target is actually quite easy, the *minimal/p1* Target Directory provided with this tutorial is (as the name suggests) very minimal! – it supports only one target (a default LMM target) and that target only supports one plugin (our “Generic” plugin).

The required SPIN target program to do this (all targets are simply SPIN programs) is only a few lines long.

This target program file is in the *minimal/p1* subdirectory, and is called *lmmdef.t*. Examine the target program file, and note the following, which are common features of all target files:

- It defines the clock frequency and stack size, but gets the values from *Catalina_Common.spin,* which in turn gets them from *Custom_DEF.inc*. Note that the stack size here refers only to the stack used during the initialization phase (not the C program stack which is not constructed till much later).

- It includes *Catalina_Common.spin*, which contains all the other platform-dependent features, such as I/O pin definitions etc.
- It includes *Catalina.spin*. This is the actual C code output by the Catalina compiler, wrapped up in a SPIN object. This file is generated on each compile, and (usually) deleted at the end of the compile (if you want to see its contents, include the **–u** flag when invoking the Catalina compiler).
- It includes *Catalina_LMM.spin*, which is the LMM Kernel.
- It includes all the plugins it might have to load (in this case, there is only the one – the generic plugin defined in *Catalina_Plugin.spin*, which is loaded if the **PLUGIN** symbol has been defined).
- It contains one method (**Start**) which initializes the registry, allocates any Hub RAM required, loads and starts all the plugins, and then invokes the LMM kernel.
- It includes *Extras.spin* file, which is described in the next section, and calls *Extras.Setup* and *Extras.Start*.

Setting up and Starting Plugins

Since Catalina itself uses plugins internally, plugins can be loaded and started in various ways, triggered by various conditions within the Catalina target files. But the most common (and recommended) way to load and start user-defined plugins is to use the **Extras.spin** file in each target directory.

Using this file means that in target directories containing multiple targets, only one file ever needs to be edited to add the new plugin to *all* targets. The simplest example of an **Extras.spin** file is the one in the *minimal/p1* subdirectory.

The essential part of this **Extras.Spin** file is as shown below:

```
'
' Select the appropriate plugin objects:
'
OBJ
  Common : "Catalina_Common"

#ifdef PLUGIN
  PLUGIN   : "Catalina_Plugin"       ' Generic Plugin
#endif

'
' This function is called by the target to allocate data blocks or set up
' other details for the extra plugins. If it does not need to do anything
' it can simply be a null routine.
'
PUB Setup
  ' Setup and start the Plugin if the PLUGIN symbol is defined
#ifdef PLUGIN
  ' Setup the plugin
  PLUGIN.Setup
#endif

'
' This function will be called by the targets to start the plugins:
'
PUB Start
  ' Setup and start the Plugin if the PLUGIN symbol is defined
#ifdef PLUGIN
  ' Start the plugin
  PLUGIN.Start
#endif
```

This illustrates how a plugin should be loaded and started if it is enabled by a particular command-line symbol (in this case **PLUGIN**).

By convention, each plugin should provide two public Spin methods:

Setup This method should perform all initial setup for the plugin. All the **Setup** methods for all the plugins are called before any of the **Start** functions are called.

Start This method should load the plugin into cog ram and start it, and then perform any subsequent initialization required.

All the loaded plugin's **Setup** functions should be called by **Extras.Setup**, and all the loaded plugin's **Start** functions will be called by **Extras.Start**.

If a plugin requires any Hub RAM allocated to it, this should be done in the plugin's **Setup** method, and allocated *downwards* from the current **FREE_MEM** pointer (which should be updated accordingly). The C program stack will be constructed starting just *below* any Hub RAM allocated to the plugins (or for the registry). This is different to when a plugin is started dynamically by a C program, where any Hub RAM required will have to be allocated in global RAM, or perhaps on the C stack. Plugins should not assume any particular Hub RAM locations (apart from a very few special cases where data blocks are permanently allocated for specific plugins in the upper Hub RAM area).

Advanced Concepts

EMM & XMM Targets

The *minimal\p1* subdirectory provided as an example Target Directory only provides a single LMM target. While the plugin code itself is often identical for EMM, XMM and LMM targets, the kernel loaded, and the means of loading and initializing the program and plugins, may differ.

Refer to the standard Catalina target directory (*target\p1*) or the embedded Catalina target directory (*embedded\p1*) for examples of more complex targets, and how to load plugins for other targets.

Supporting multiple Propeller platforms

The *Catalina_Common.spin* file provided in the *minimal\p1* subdirectory only supports a single platform. To support different platforms, a unique symbol is first selected that can be defined on the command line, and the code for that platform is wrapped in conditional compilation constructs – i.e. by using constructs like:

```
#ifdef CUSTOM_1
   ...
#elseifdef CUSTOM_2
   ...
#endif.
```

The first step in supporting a new platform is to determine a suitable symbol for it (in this case **CUSTOM_1** or **CUSTOM_2**) and then use it consistently in all target files and plugins.

For an example of this, see the version of *Catalina_Common.spin* in the default target (in the directory *Catalina\target*). This version of the file includes *DEF.inc,* which contains a construct like the one shown above which specifies all supported platforms.

